Pirate Intelligent Agent Design Defense

Jennifer Joseph

Southern New Hampshire University

CS 370: Current/Emerging Trend in CS

Matthew McHann

8/3/2026

Pirate Intelligent Agent Design Defense

A human solving the treasure maze would probably begin by looking at the available paths and identifying which directions appear to move closer to the treasure. The person would avoid walls and dead ends when possible. If a selected path failed, the person could backtrack and try another route. After several attempts, previous mistakes would make it easier to recognize which paths should be avoided. A human can use visual reasoning, memory, and prior experience to decide what move seems most useful at any point in the maze.

The intelligent agent solves the problem differently because it cannot look at the maze and reason about it in the same way a person can. Instead, the pirate learns through reinforcement learning. It begins at different locations in the maze, selects an available action, receives a reward or penalty, and uses the result to improve future decisions. Q-learning is based on learning the expected value of actions and using those values to select actions that are more likely to lead to a greater future reward (Violante, 2019).

There are still similarities between the two approaches. Both improve by using the results of previous decisions. A person remembers that a route led to a dead end, while the intelligent agent stores states, actions, rewards, and results as experience. Both can also try an unfamiliar route when the best solution is not known. The main difference is how those decisions are made. Human reasoning uses visual information and judgment, while the pirate relies on numerical rewards and Q-values generated by the neural network.

The purpose of the intelligent agent is to learn a reliable path through the maze so the pirate can reach the treasure. Reinforcement learning is useful for this problem because the program does not need to be given the correct sequence of moves for every possible starting position. Instead, the agent learns from repeated interaction with the environment. Successful

actions receive useful rewards, while actions that lead to poor results are less likely to be selected in the future.

An important part of this process is balancing exploration and exploitation. Exploration occurs when the agent tries an action without relying on the path it currently believes is best. This allows the pirate to discover routes that have not been tested. Exploitation occurs when the agent uses what it has already learned and chooses the action with the highest predicted Q-value. Too much exploration would cause the pirate to continue making unnecessary random moves, while too much exploitation early in training could cause it to repeatedly use a poor route before discovering a better one (Violante, 2019).

The implementation uses an exploration factor called epsilon. Training begins with an epsilon value of 1.0, so the pirate initially explores the maze heavily. Epsilon gradually decreases using a decay rate of 0.995. Once the win rate is above 90%, epsilon is set to 0.05. At that stage, the agent is effectively using approximately 5% exploration and 95% exploitation. This is a reasonable balance for this pathfinding problem because exploration is important while the maze is still being learned, but exploitation should dominate after the agent has found reliable routes. A small amount of continued exploration prevents the model from becoming completely dependent on only one sequence of previously successful decisions.

Reinforcement learning ultimately allows the pirate to connect actions with their long-term results. Instead of simply moving toward the treasure based on distance, it learns which actions from different states are most likely to produce a successful outcome. This allows the same trained agent to navigate the maze from different starting positions.

Deep Q-learning combines Q-learning with an artificial neural network. Traditional Q-learning can store values in a Q-table, but a neural network can estimate Q-values from the

current state instead. Neural networks are able to learn relationships in data by adjusting weighted connections during training (Gulli & Pal, 2017). In this project, the network receives the current maze state and produces values for the four available actions: left, up, right, and down. The action with the highest predicted value can then be selected during exploitation.

The model contains two hidden dense layers followed by an output layer containing the values for the possible actions. PReLU activation functions are used between the hidden layers. Training uses the Adam optimizer and mean squared error to measure the difference between the expected target values and the values predicted by the model. The weights are adjusted during training so that the predicted Q-values become more accurate.

Experience replay is also used during training. After the pirate takes an action, the previous state, action, reward, new state, and game status are stored. Samples of these previous experiences are then reused to train the neural network. Reusing stored experiences allows the network to learn from more than just the most recent move and reduces its dependence on a single sequence of events. This type of repeated neural network training helps the model identify useful patterns from previous experiences (Gulli & Pal, 2017).

The implementation also uses a target network. The target model begins with the same weights as the primary model and is updated periodically during training. Separating the target network from the model that is currently being trained provides more stable target values while the primary network changes. The completed model eventually reached a 1.000 win rate at epoch 495. The completion check also returned true, which was important because a 100% recent win rate alone does not guarantee that the pirate can successfully navigate from every required starting position. The final result shows that deep Q-learning was able to learn the maze well enough to consistently guide the intelligent agent to the treasure.

References

Gulli, A., & Pal, S. (2017). Deep learning with Keras. Packt Publishing. https://www.packtpub.com/en-GB/product/deep-learning-with-keras-9781787128422

Violante, A. (2019, March 18). Simple reinforcement learning: Q-learning. Towards Data Science. https://medium.com/data-science/simple-reinforcement-learning-q-learning-fcddc4b6fe56